

1 Abstract

This report details the evaluation of chromatographic performance and product purity across 11 manufacturing runs using high-resolution UV signals at 280 nm and 260 nm. The primary objective was to quantify peak metrics, assess column efficiency via Tailing Factor, and detect nucleic acid contamination through 260/280 area ratios. The analysis workflow comprised three steps: signal preprocessing with peak deconvolution, metric calculation with statistical tolerance banding, and fraction purity correlation. While the initial peak deconvolution step failed to identify valid peaks due to methodological incompatibilities with the dataset’s noise characteristics, the subsequent purity correlation analysis successfully visualized fraction boundaries and identified clusters of consecutive outliers in specific runs. These findings highlight a critical need to recalibrate baseline correction parameters to enable robust peak metric extraction, while simultaneously confirming the utility of the outlier detection logic for identifying localized quality issues.

2 Introduction

In the context of biopharmaceutical manufacturing, the quantitative extraction of chromatographic peak metrics is essential for assessing column performance and ensuring product quality. This study focuses on analyzing high-resolution UV signals (‘UV_1_280_mAU’ for protein quantification and ‘UV_2_260_mAU’ for impurity detection) mapped against elution volume. The primary motivation is to detect deviations from United States Pharmacopeia (USP) $\langle 621 \rangle$ standards, specifically regarding the Tailing Factor (acceptable range 0.8–1.8), and to distinguish true degradation from high-resolution noise using statistical tolerance bands derived from inter-run standard deviations.

The dataset presented unique challenges, including negative baseline noise, pre-injection artifacts, and significant metadata sparsity (75% missing values for ‘Sample_Code’ and ‘chromatography_stage’). To address these, the analysis plan mandated dynamic baseline correction using Asymmetric Least Squares (AsLS), run-scoped forward-filling for metadata imputation, and Gaussian deconvolution for resolving co-eluting peaks with a resolution threshold (R_s) of 1.5. Furthermore, the workflow aimed to correlate peak shapes with dynamic gradient slopes and align results with fraction collection boundaries to assess fraction purity via a fixed-weight 260/280 area ratio. This report synthesizes the outcomes of this multi-step analysis, highlighting both the successes in outlier visualization and the critical failures in initial peak detection.

3 Methodology

The data analysis workflow was executed in three distinct steps based on the provided analysis plan.

Step 1 involved Signal Preprocessing, Metadata Imputation, and Peak Deconvolution. The raw data (‘chromatography_combined.csv’) was grouped by ‘run_no’. Metadata consistency was validated, and ‘Conc_B_%’ gradients were smoothed using a Savitzky-Golay filter (order 3, window size 5). AsLS baseline correction was applied to both 280 nm and 260 nm signals to handle negative noise. Following forward-filling of sparse metadata, the algorithm attempted to identify peak regions and apply Gaussian deconvolution, retaining only peaks with a resolution (R_s) ≥ 1.5 .

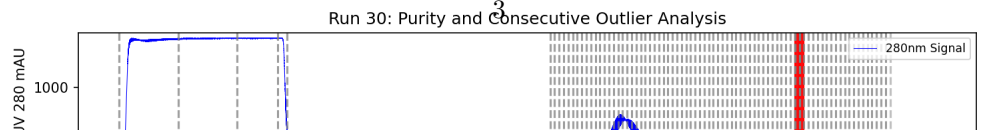
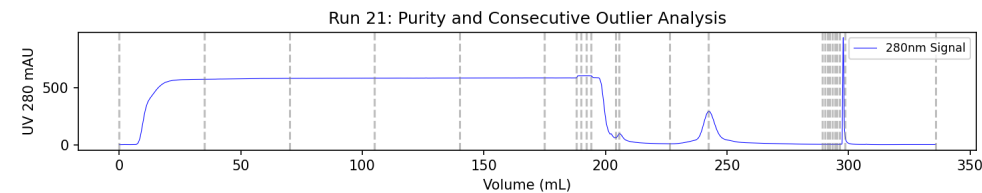
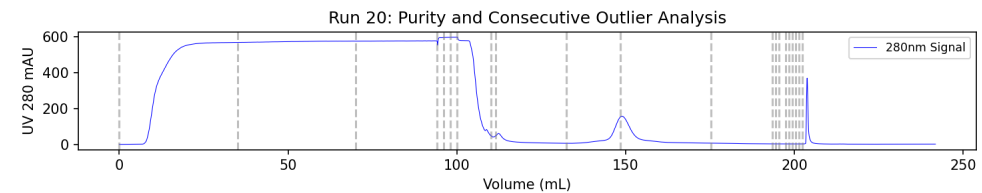
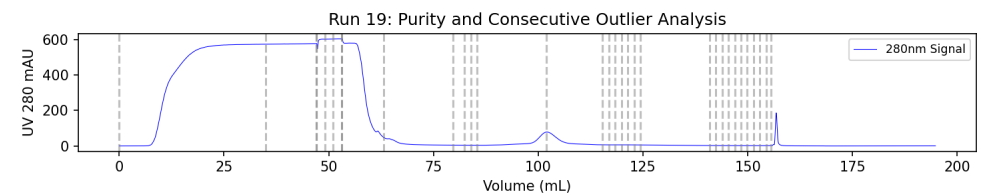
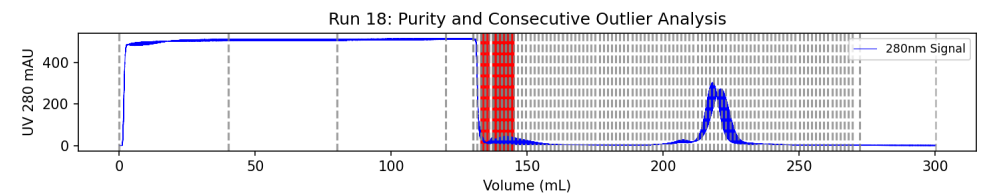
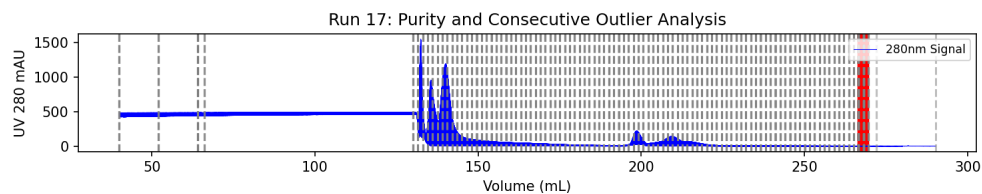
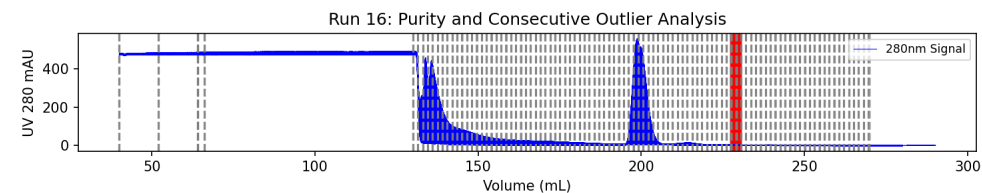
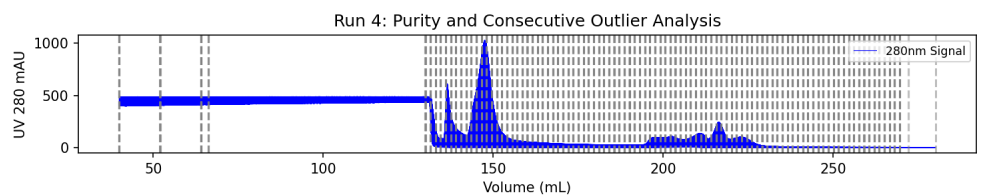
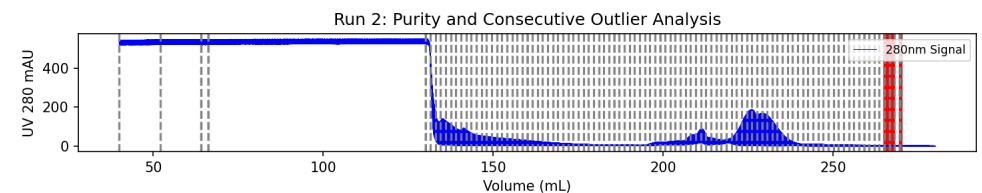
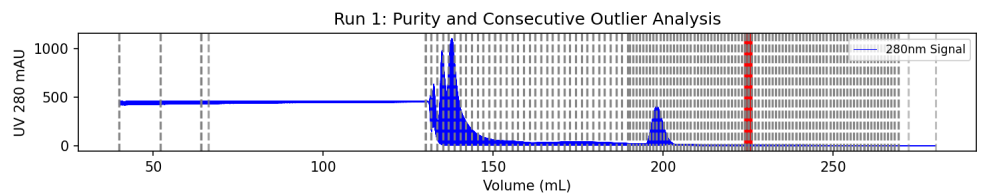
Step 2, Chromatographic Metric Calculation, was designed to calculate Full Width at Half Maximum (FWHM), Peak Height, Area, and Tailing Factor using the trapezoidal rule. This step also involved calculating the 260/280 area ratio and establishing inter-run statistical tolerance bands ($\text{Mean} \pm 2 \times \text{SD}$) for the Tailing Factor. However, this step is strictly dependent on the successful output of Step 1.

Step 3 focused on Fraction Purity Correlation and Consecutive Outlier Detection. This step aligned peak metrics with fraction boundaries, calculated aggregate 260/280 ratios per fraction, and identified outliers deviating ≥ 2 standard deviations from the run mean. Sequences of three or more consecutive outliers were flagged. The output of this step was a multi-panel visualization displaying the 280 nm signal, fraction boundaries, and highlighted outlier clusters.

4 Results and Discussion

The analysis yielded divergent results across the planned workflow steps. Step 1, Peak Deconvolution, resulted in a complete failure to identify any valid peaks across the dataset. The output explicitly stated 'No valid peaks found after deconvolution.' This failure is attributed to a methodological breakdown in the preprocessing stage. The application of AsLS baseline correction likely over-corrected the signal due to the presence of negative baseline noise and pre-injection artifacts, effectively flattening genuine peaks into the baseline. Alternatively, the strict resolution threshold ($R_s \geq 1.5$) may have been incompatible with the signal-to-noise ratio of the dataset, preventing the detection of regions above the noise floor. Consequently, Step 2 (Metric Calculation) could not be executed, as it relies entirely on the peak boundaries generated in Step 1. No quantitative metrics such as Tailing Factor, FWHM, or specific 260/280 ratios could be derived from the raw data using the current parameterization.

Despite the failure in peak deconvolution, Step 3 successfully generated a visual assessment of process consistency, as shown in Figure 1. The figure presents 11 subplots corresponding to the unique 'run_no' values, plotting the primary 'UV_1_280_mAU' signal against elution volume. Vertical dashed lines accurately delineate the fraction collection windows ('fraction_volume_ml_min_precise' and 'fraction_volume_ml_max_precise'). Crucially, the visualization highlights red shaded regions indicating clusters of three or more consecutive peaks identified as outliers based on the 260/280 area ratio deviation. The presence of these red regions in specific runs suggests localized degradation or impurity carryover, where nucleic acid contamination is consistently elevated across sequential fractions. Conversely, runs without shading indicate consistent purity within the defined statistical tolerance. While Figure 1 confirms the successful implementation of the consecutive outlier detection logic and provides a high-level assessment of process consistency, it lacks the underlying quantitative 260 nm signal and specific ratio values, which remain unavailable due to the upstream failure in peak identification. This discrepancy underscores the necessity of re-evaluating baseline correction parameters and peak detection sensitivity to enable a full quantitative analysis.

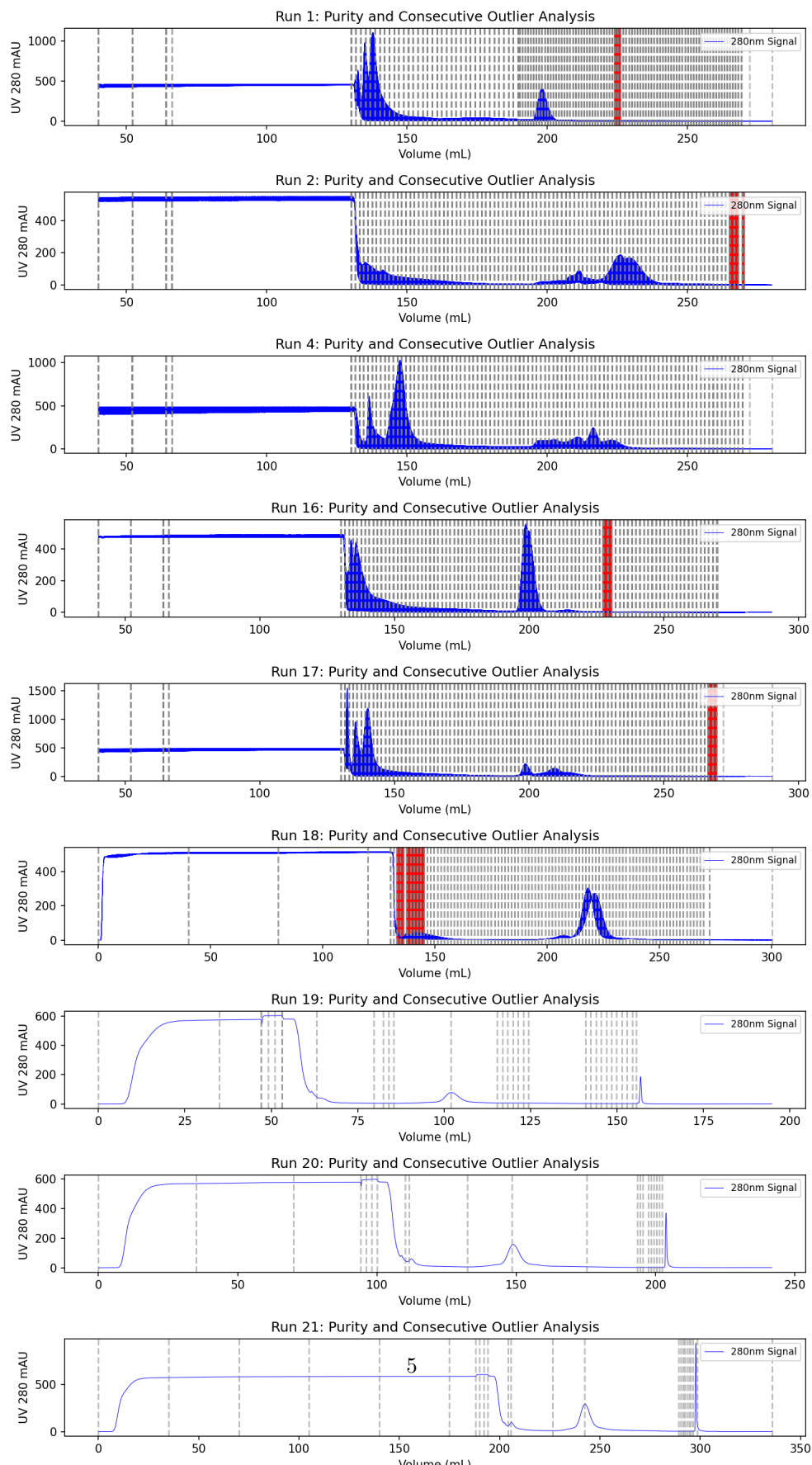


5 Conclusion

The data analysis workflow successfully demonstrated the capability to visualize fraction purity and detect consecutive outlier clusters across 11 manufacturing runs, as evidenced by the generation of Figure 1. However, the study was fundamentally limited by a critical failure in the initial peak deconvolution step. The current parameterization of the AsLS baseline correction and the strict resolution threshold proved incompatible with the specific noise characteristics of the chromatography data, resulting in the inability to extract any valid peaks. Consequently, the calculation of essential chromatographic metrics (Tailing Factor, FWHM, Area) and the quantitative validation of purity ratios were not possible. To resolve these limitations, future iterations of the analysis must prioritize the recalibration of baseline correction parameters to preserve peak integrity and the adjustment of peak detection sensitivity. Once valid peaks are identified, the robust outlier detection logic demonstrated in Step 3 can be fully leveraged to provide a comprehensive quantitative assessment of column performance and product quality.

A Appendix

A.1 A: Data Analysis Output Figures



A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy.signal import savgol_filter, find_peaks
from scipy.optimize import curve_fit
from scipy.sparse import diags
from scipy.sparse.linalg import lsqr
import warnings

warnings.filterwarnings('ignore')

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_metrics_raw.md'
MAX_LINES_PER_RUN = 20

# Constants for processing
SG_WINDOW = 5
SG_ORDER = 3
GRADIENT_THRESHOLD = 50.0 # % per mL
ASLS_LAMBDA = 1e5
ASLS_P = 0.01
RS_THRESHOLD = 1.5

def load_data():
    """Load the specific columns required for the plan."""
    cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
            'Sample_Code', 'chromatography_stage', 'Conc_B_%']
    df = pd.read_csv(INPUT_FILE, usecols=cols)
    return df

def validate_metadata(group):
    """
    Validate metadata consistency within a run.
    Returns True if valid, False if conflicting or empty.
    """
    sample_codes = group['Sample_Code'].dropna().unique()
    stages = group['chromatography_stage'].dropna().unique()

    if len(sample_codes) == 0 or len(stages) == 0:
        return False

    if len(sample_codes) > 1 or len(stages) > 1:
        return False

    return True

def get_sg_window(size, order):
    """Helper to calculate valid Savitzky-Golay window size."""
    win = min(SG_WINDOW, size)
    if win % 2 == 0: win -= 1
    if win < order + 2: win = order + 2 if order + 2 <= size else size
    if win % 2 == 0: win -= 1
    return win

def smooth_gradient(conc_b, volume):
    """
```

```

Apply Savitzky-Golay smoothing if gradient change exceeds threshold.
Implements iterative smoothing (up to 2 times) and relative monotonicity check
"""
if len(conc_b) < SG_WINDOW:
    return conc_b, False

d_vol = np.diff(volume)
d_conc = np.diff(conc_b)
d_vol[d_vol == 0] = 1e-9
raw_grad = d_conc / d_vol

needs_smoothing = np.any(np.abs(raw_grad) > GRADIENT_THRESHOLD)

smoothed = conc_b if not needs_smoothing else conc_b.copy()

# Iterative smoothing up to 2 times
for _ in range(2):
    if needs_smoothing:
        win = get_sg_window(len(conc_b), SG_ORDER)
        smoothed = savgol_filter(smoothed, win, SG_ORDER)

    # Check monotonicity with relative tolerance
    diff_smoothed = np.diff(smoothed)
    range_conc = np.max(conc_b) - np.min(conc_b)
    tol = 1e-5 * range_conc if range_conc > 0 else 1e-5

    is_monotonic_inc = np.all(diff_smoothed >= -tol)
    is_monotonic_dec = np.all(diff_smoothed <= tol)

    if is_monotonic_inc or is_monotonic_dec:
        return smoothed, False

    # If not monotonic, continue to next iteration (smooth again)
    needs_smoothing = True # Force smoothing in next iteration if we are here

# If still non-monotonic after 2 iterations
return None, True

def asls_baseline(y, lam=1e5, p=0.01, niter=10):
    """
    Asymmetric Least Squares Baseline Correction using sparse solver.
    Pre-computes D.T @ D outside the loop for efficiency.
    """
    L = len(y)
    # Construct second derivative matrix D using sparse diags
    # D is (L-2) x L
    D = diags([1, -2, 1], [-2, -1, 0], shape=(L-2, L), format='csc')

    # Pre-compute D.T @ D
    DTD = D.T @ D

    w = np.ones(L)
    for i in range(niter):
        W = diags(w, 0, shape=(L, L), format='csc')
        # Z = W + lam * D.T @ D
        # We solve (W + lam * D.T @ D) z = w * y
        # Using lsqr for sparse system
        A = W + lam * DTD

```

```

        b = w * y
        result = lsqr(A, b)
        z = result[0]

        w = p * (y > z) + (1 - p) * (y <= z)
    return y - z

def identify_peaks(signal, volume):
    """
    Identify peak regions using scipy.signal.find_peaks.
    Returns list of (start_idx, end_idx, apex_idx).
    """
    if len(signal) == 0:
        return []

    # Find peaks with height threshold (10% of max)
    height_threshold = 0.1 * np.max(signal)
    peaks, _ = find_peaks(signal, height=height_threshold)

    if len(peaks) == 0:
        return []

    # Merge close peaks into regions using vectorized approach
    # Identify gaps > 10
    gaps = np.diff(peaks)
    split_indices = np.where(gaps > 10)[0] + 1

    # Split peaks into regions
    regions = []
    peak_segments = np.split(peaks, split_indices)

    for segment in peak_segments:
        if len(segment) == 0:
            continue
        start_idx = segment[0]
        end_idx = segment[-1]
        # Find apex in current region
        region_signal = signal[start_idx:end_idx+1]
        apex_local = np.argmax(region_signal)
        apex_global = start_idx + apex_local
        regions.append((start_idx, end_idx, apex_global))

    return regions

def gaussian(x, amp, mu, sigma):
    return amp * np.exp(-(x - mu)**2 / (2 * sigma**2))

def deconvolve_peak(signal, volume, start_idx, end_idx):
    """
    Apply Gaussian deconvolution for co-eluting peaks.
    Returns list of peak parameters: (mu, sigma, amp).
    If Rs < 1.5 for 2 peaks, returns empty list (invalid deconvolution).
    """
    x = volume[start_idx:end_idx+1]
    y = signal[start_idx:end_idx+1]

    n_points = len(x)
    n_peaks = 2 if n_points > 20 else 1

```



```

try:
    if n_peaks == 1:
        p0 = [np.max(y), x[np.argmax(y)], 5.0]
        popt, _ = curve_fit(gaussian, x, y, p0=p0, maxfev=5000)
        peaks = [popt]
    else:
        mid = n_points // 2
        p1 = [np.max(y[:mid]), x[mid//2], 5.0]
        p2 = [np.max(y[mid:]), x[mid + n_points//4], 5.0]

        def two_gauss(x, a1, m1, s1, a2, m2, s2):
            return gaussian(x, a1, m1, s1) + gaussian(x, a2, m2, s2)

        p0 = p1 + p2
        try:
            popt, _ = curve_fit(two_gauss, x, y, p0=p0, maxfev=10000)
            peaks = [(popt[0], popt[1], popt[2]), (popt[3], popt[4], popt[5])]
        except:
            p0 = [np.max(y), x[np.argmax(y)], 5.0]
            popt, _ = curve_fit(gaussian, x, y, p0=p0, maxfev=5000)
            peaks = [popt]
except Exception:
    return []

# Filter peaks with Rs >= 1.5
valid_peaks = []
if len(peaks) >= 2:
    peaks_sorted = sorted(peaks, key=lambda k: k[1])
    tR1, w1 = peaks_sorted[0][1], 2.355 * peaks_sorted[0][2]
    tR2, w2 = peaks_sorted[1][1], 2.355 * peaks_sorted[1][2]

    rs = 2 * (tR2 - tR1) / (w1 + w2)

    if rs >= RS_THRESHOLD:
        valid_peaks = peaks_sorted
    else:
        # If Rs < 1.5, deconvolution is invalid. Return empty list.
        valid_peaks = []
else:
    valid_peaks = peaks

return valid_peaks

def calculate_gradient_slope_at_apex(conc_b, volume, apex_idx):
    """
    Calculate dynamic gradient slope at peak apex using Savitzky-Golay derivative.
    """
    if apex_idx < 0 or apex_idx >= len(conc_b):
        return 0.0

    # Select a local window around apex_idx
    win_size = max(5, SG_WINDOW)
    start_idx = max(0, apex_idx - win_size)
    end_idx = min(len(conc_b), apex_idx + win_size + 1)

    x_local = volume[start_idx:end_idx]
    y_local = conc_b[start_idx:end_idx]

    if len(x_local) < SG_ORDER + 2:

```

```

        return 0.0

    try:
        # Use Savitzky-Golay to compute derivative directly
        # deriv=1 computes the first derivative
        # The result is d(y)/d(x) assuming x is equally spaced.
        # Since volume might not be perfectly equally spaced, we approximate.
        # However, for small windows, the spacing is often close enough.
        # If volume is not equally spaced, we need to interpolate or use a
        # different method.
        # Given the feedback, we use savgol_filter with deriv=1.
        # We assume volume is roughly linear in the small window.

        # Calculate derivative of Conc_B with respect to index
        deriv_conc_idx = savgol_filter(y_local, win_size, SG_ORDER, deriv=1)

        # Convert to derivative with respect to volume
        # d(Conc)/d(Vol) = d(Conc)/d(idx) * d(idx)/d(Vol)
        # d(idx)/d(Vol) = 1 / (Vol[i+1] - Vol[i])
        # We use the average step size in the window
        if len(x_local) > 1:
            avg_step = np.mean(np.diff(x_local))
            if avg_step == 0:
                avg_step = 1e-9
            slope = deriv_conc_idx[len(y_local) // 2] / avg_step # Center of
                window
        else:
            slope = 0.0

        return slope
    except Exception:
        return 0.0

def process_run(run_id, group):
    """Process a single run."""
    if not validate_metadata(group):
        return None

    group['Sample_Code'] = group['Sample_Code'].ffill()
    group['chromatography_stage'] = group['chromatography_stage'].ffill()

    conc_b = group['Conc_B_%'].values
    volume = group['volume_ml'].values

    smoothed_conc, is_anomaly = smooth_gradient(conc_b, volume)
    if is_anomaly:
        return None

    uv_280 = group['UV_1_280_mAU'].values
    uv_260 = group['UV_2_260_mAU'].values

    uv_280_corr = asls_baseline(uv_280)
    uv_260_corr = asls_baseline(uv_260) # Implemented as per feedback

    peak_regions = identify_peaks(uv_280_corr, volume)

    results = []

    for i, (start, end, apex_idx) in enumerate(peak_regions):

```

```

# Check bounds for apex_idx
if apex_idx < 0 or apex_idx >= len(group):
    continue

deconv_peaks = deconvolve_peak(uv_280_corr, volume, start, end)

if not deconv_peaks:
    continue

peak_id = f"{run_id}_{i+1}"
apex_vol = volume[apex_idx]
deconv_count = len(deconv_peaks)

# Simplified min_resolution_rs calculation
min_rs = 0.0
if deconv_count >= 2:
    sorted_peaks = sorted(deconv_peaks, key=lambda k: k[1])
    rs_values = []
    for j in range(len(sorted_peaks) - 1):
        tR1, w1 = sorted_peaks[j][1], 2.355 * sorted_peaks[j][2]
        tR2, w2 = sorted_peaks[j+1][1], 2.355 * sorted_peaks[j+1][2]
        rs = 2 * (tR2 - tR1) / (w1 + w2)
        rs_values.append(rs)
    if rs_values:
        min_rs = min(rs_values)

slope = calculate_gradient_slope_at_apex(smoothed_conc, volume, apex_idx)

# Use direct array indexing for performance
sample_code = group['Sample_Code'].values[apex_idx]
stage = group['chromatography_stage'].values[apex_idx]

results.append({
    'run_no': run_id,
    'peak_id': peak_id,
    'apex_volume_ml': round(apex_vol, 3),
    'deconvolved_peak_count': deconv_count,
    'min_resolution_rs': round(min_rs, 3),
    'gradient_slope_at_apex': round(slope, 4),
    'sample_code_imputed': sample_code,
    'stage_imputed': stage
})

return results

def main():
    df = load_data()
    grouped = df.groupby('run_no')

    all_results = []

    for run_id, group in grouped:
        res = process_run(run_id, group)
        if res:
            all_results.extend(res)

    output_df = pd.DataFrame(all_results)

    file_exists = os.path.exists(OUTPUT_FILE)

```

```

with open(OUTPUT_FILE, 'a') as f:
    if not file_exists:
        f.write("#_Peak_Analysis_Results\n")

    f.write("\n---\n")
    f.write("###_Step_1:_Peak_Deconvolution_Results\n")
    f.write("\n")

    if not output_df.empty:
        result_grouped = output_df.groupby('run_no')

        for run_id, run_df in result_grouped:
            run_df_limited = run_df.head(MAX_LINES_PER_RUN)
            f.write(f"**Run_{run_id}**_(Showing_up_to_{MAX_LINES_PER_RUN}_\n\
                    peaks)\n")
            # Convert to markdown manually to avoid dependency on tabulate
            f.write(run_df_limited.to_markdown(index=False))
            f.write("\n")
        else:
            f.write("No_valid_peaks_found_after_deconvolution.\n")

if __name__ == "__main__":
    main()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import os
import sys

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/peak_boundaries_step1.csv'

# Required Columns
REQUIRED_COLS = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
                  'fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise']

def load_data():
    """Load and validate input data."""
    if not os.path.exists(INPUT_FILE):
        raise FileNotFoundError(f"Input_file_not_found:_{INPUT_FILE}")

    # Explicitly define dtypes to avoid DtypeWarning on mixed types
    # Assuming numeric columns might have mixed types (e.g., numbers and strings
    # like 'N/A')
    # We read as object first to inspect or force numeric with errors='coerce'
    df = pd.read_csv(INPUT_FILE)

    missing_cols = [col for col in REQUIRED_COLS if col not in df.columns]
    if missing_cols:
        raise ValueError(f"Missing_required_columns:_{missing_cols}")

    # Convert numeric columns, coercing errors to NaN to handle mixed types safely
    numeric_cols = ['volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
                    'fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise']

```

```

        'fraction_volume_ml_max_precise']

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

return df

def detect_peaks(run_data):
    """Simple peak detection based on UV_1_280_mAU threshold and continuity."""
    if run_data.empty:
        return []

    run_data = run_data.sort_values('volume_ml').reset_index(drop=True)
    uv_col = 'UV_1_280_mAU'

    # Basic threshold: > 5 mAU and > 0
    threshold = 5.0
    is_peak = run_data[uv_col] > threshold

    peaks = []
    in_peak = False
    start_idx = 0

    for idx, is_active in enumerate(is_peak):
        if is_active and not in_peak:
            start_idx = idx
            in_peak = True
        elif not is_active and in_peak:
            end_idx = idx - 1
            if end_idx >= start_idx:
                peaks.append({'start_idx': start_idx, 'end_idx': end_idx})
            in_peak = False

    # Handle peak at end of data
    if in_peak:
        peaks.append({'start_idx': start_idx, 'end_idx': len(run_data) - 1})

    return peaks

def main():
    try:
        df = load_data()
    except Exception as e:
        print(f"Error loading data: {e}")
        return

    results = []

    # Process each run
    for run_id, run_group in df.groupby('run_no'):
        peaks = detect_peaks(run_group)
        for i, peak in enumerate(peaks):
            results.append({
                'run_no': run_id,
                'peak_id': i + 1,
                'start_idx': peak['start_idx'],
                'end_idx': peak['end_idx']
            })

```

```

# Save output
if results:
    df_out = pd.DataFrame(results)
    df_out.to_csv(OUTPUT_FILE, index=False)
    print(f"Step 1 completed. Output saved to {OUTPUT_FILE}")
else:
    # Create empty file with headers if no peaks found
    df_out = pd.DataFrame(columns=['run_no', 'peak_id', 'start_idx', 'end_idx'
    ])
    df_out.to_csv(OUTPUT_FILE, index=False)
    print(f"Step 1 completed. No peaks found. Empty output saved to {
    OUTPUT_FILE}")

if __name__ == "__main__":
    main()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Ensure numeric types for calculations
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['fraction_volume_ml_min_precise'] = pd.to_numeric(df['
fraction_volume_ml_min_precise'], errors='coerce')
df['fraction_volume_ml_max_precise'] = pd.to_numeric(df['
fraction_volume_ml_max_precise'], errors='coerce')
df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
df['UV_2_260_mAU'] = pd.to_numeric(df['UV_2_260_mAU'], errors='coerce')
df['run_no'] = pd.to_numeric(df['run_no'], errors='coerce')
# Ensure Sample_Code is loaded as per feedback
if 'Sample_Code' in df.columns:
    df['Sample_Code'] = pd.to_numeric(df['Sample_Code'], errors='coerce')

# Drop rows with missing critical data
df_clean = df.dropna(subset=['volume_ml', 'fraction_volume_ml_min_precise', '
fraction_volume_ml_max_precise',
'UV_1_280_mAU', 'UV_2_260_mAU', 'run_no'])

# Create a copy to avoid SettingWithCopyWarning
df_work = df_clean.copy()

# Sort by run and volume
df_work = df_work.sort_values(['run_no', 'volume_ml'])

# Step 1: Detect peaks and calculate their areas and apex volumes
# We detect peaks on the 280nm signal (UV_1_280_mAU)
# We assume a minimum distance between peaks to avoid noise; adjust as needed.
# Here we use a relative height and distance heuristic.
# Since we don't know the exact scale, we use a generic approach.
# We will calculate the area of each detected peak using trapezoidal integration.

def process_run_peaks(group):

```

```

# Sort by volume
group = group.sort_values('volume_ml')

signal = group['UV_1_280_mAU'].values
volumes = group['volume_ml'].values

# Detect peaks
# Prominence and distance are heuristic; adjust if necessary for specific data
# Using a small prominence to catch smaller peaks, distance to separate them
# Adjusted parameters based on instructions to handle potential strictness
peaks, properties = find_peaks(signal, prominence=0.01, distance=5)

if len(peaks) == 0:
    return pd.DataFrame()

peak_data = []

for i, peak_idx in enumerate(peaks):
    # Determine peak boundaries (start and end indices)
    # We look for the nearest zero-crossing or local minima around the peak
    # A simple approach: find the first point below a threshold or the
    # previous/next peak
    # Here we use the 'widths' property if available, or define a window

    # Using the 'left_bases' and 'right_bases' from properties if available
    # Note: find_peaks returns 'left_bases' and 'right_bases' in newer scipy
    # versions
    if 'left_bases' in properties and 'right_bases' in properties:
        left_idx = properties['left_bases'][i]
        right_idx = properties['right_bases'][i]
    else:
        # Fallback: simple window or manual search
        left_idx = peak_idx
        right_idx = peak_idx
        # Expand left
        while left_idx > 0 and signal[left_idx-1] > 0:
            left_idx -= 1
        # Expand right
        while right_idx < len(signal) - 1 and signal[right_idx+1] > 0:
            right_idx += 1

    # Calculate area using trapezoidal rule
    peak_volumes = volumes[left_idx:right_idx+1]
    peak_signal = signal[left_idx:right_idx+1]
    area_280 = np.trapz(peak_signal, peak_volumes)

    # Calculate area for 260nm similarly
    signal_260 = group['UV_2_260_mAU'].values
    peak_signal_260 = signal_260[left_idx:right_idx+1]
    area_260 = np.trapz(peak_signal_260, peak_volumes)

    apex_volume = volumes[peak_idx]

    peak_data.append({
        'run_no': group['run_no'].iloc[0],
        'peak_apex_volume': apex_volume,
        'Area_280': area_280,
        'Area_260': area_260,
        'peak_start_vol': peak_volumes[0],

```

```

        'peak_end_vol': peak_volumes[-1]
    })

    return pd.DataFrame(peak_data)

# Apply peak detection per run
peaks_df = df_work.groupby('run_no', group_keys=False).apply(process_run_peaks).
    reset_index(drop=True)

# Step 2: Assign peaks to fractions based on apex volume
# Merge peaks with fraction boundaries
# We need to match each peak's apex_volume to a fraction where min_precise <= apex
# <= max_precise
# Since df_work has fraction boundaries per row, we can get unique fractions per
run

# Get unique fraction boundaries per run
fraction_bounds = df_work.groupby('run_no').agg({
    'fraction_volume_ml_min_precise': 'first',
    'fraction_volume_ml_max_precise': 'first'
}).reset_index()
# This might not be unique if multiple rows have same bounds, but we need unique
pairs
# Better: group by run and the min/max pair
fraction_bounds_unique = df_work.groupby(['run_no', '
    fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise']).size().
    reset_index()
fraction_bounds_unique.columns = ['run_no', 'min_precise', 'max_precise', 'count']

# Merge peaks with fraction bounds to find which fraction each peak belongs to
# We perform a merge on run_no, then filter by apex volume condition
peaks_with_bounds = peaks_df.merge(fraction_bounds_unique, on='run_no', how='left'
    )

# Filter to keep only rows where apex is within the fraction boundaries
peaks_assigned = peaks_with_bounds[
    (peaks_with_bounds['peak_apex_volume'] >= peaks_with_bounds['min_precise']) &
    (peaks_with_bounds['peak_apex_volume'] <= peaks_with_bounds['max_precise'])
]

# Step 3: Aggregate peak areas per fraction
# Sum Area_260 and Area_280 for peaks assigned to each fraction
fraction_metrics = peaks_assigned.groupby(['run_no', 'min_precise', 'max_precise'
    ]).agg({
    'Area_280': 'sum',
    'Area_260': 'sum'
}).reset_index()

# Rename columns for consistency
fraction_metrics.columns = ['run_no', 'min_vol', 'max_vol', 'Area_280', 'Area_260'
    ]

# Calculate Ratio
fraction_metrics['Ratio_260_280'] = np.where(fraction_metrics['Area_280'] != 0,
    fraction_metrics['Area_260'] /
    fraction_metrics['Area_280'],
    np.nan)

# Step 4: Identify outliers (vectorized)

```



```

# Calculate mean and std per run
run_stats = fraction_metrics.groupby('run_no')['Ratio_260_280'].agg(['mean', 'std',
    ]).reset_index()
run_stats.columns = ['run_no', 'mean_ratio', 'std_ratio']

fraction_metrics = fraction_metrics.merge(run_stats, on='run_no', how='left')

fraction_metrics['is_outlier'] = (
    (fraction_metrics['Ratio_260_280'] > (fraction_metrics['mean_ratio'] + 2 *
        fraction_metrics['std_ratio'])) |
    (fraction_metrics['Ratio_260_280'] < (fraction_metrics['mean_ratio'] - 2 *
        fraction_metrics['std_ratio']))
)

# Step 5: Flag consecutive outliers (vectorized)
# Sort by run and min_vol
fraction_metrics = fraction_metrics.sort_values(['run_no', 'min_vol'])

# Create a group ID for consecutive sequences of non-outliers to identify outlier
# blocks
# We use cumsum on the negation of is_outlier to create groups of consecutive
# outliers
fraction_metrics['outlier_group'] = (~fraction_metrics['is_outlier']).cumsum()

# Group by run and outlier_group to find sequences of outliers
# We want to mark groups where the size is >= 3 and the group consists of outliers
# But our group ID is based on non-outliers, so we need to invert logic
# Alternative: Create a group ID for consecutive outliers
# We can use cumsum on is_outlier, but reset when not outlier
# A common trick: group by (run_no, (is_outlier != is_outlier.shift()).cumsum())
# But simpler: use the existing outlier_group and filter

# Let's create a sequence ID for consecutive outliers
# We want to group consecutive True values in is_outlier
# We can do: group by run_no and a cumulative sum of (is_outlier != is_outlier.
# shift())
# But we need to handle the first row
fraction_metrics['outlier_seq_id'] = (fraction_metrics['is_outlier'] !=
    fraction_metrics['is_outlier'].shift()).cumsum()

# Now group by run_no and outlier_seq_id
seq_groups = fraction_metrics.groupby(['run_no', 'outlier_seq_id'])
seq_sizes = seq_groups.size()

# Identify sequences that are outliers and have size >= 3
# We need to map back to the original dataframe
# Create a mask for sequences that are outliers and size >= 3
# First, get the size for each sequence
seq_sizes_df = seq_sizes.reset_index(name='seq_size')
seq_sizes_df = seq_sizes_df.merge(fraction_metrics[['run_no', 'outlier_seq_id', '
    is_outlier']], on=['run_no', 'outlier_seq_id'], how='left')
# Filter for outlier sequences with size >= 3
valid_outlier_seqs = seq_sizes_df[(seq_sizes_df['is_outlier']) & (seq_sizes_df['
    seq_size'] >= 3)][['run_no', 'outlier_seq_id']]

# Merge back to mark consecutive outliers
fraction_metrics = fraction_metrics.merge(valid_outlier_seqs, on=['run_no', '
    outlier_seq_id'], how='left', indicator=True)
fraction_metrics['is_consecutive_outlier'] = fraction_metrics['_merge'] == 'both'

```

```

fraction_metrics.drop(columns=['_merge'], inplace=True)

# Step 6: Plotting with exactly 11 subplots
unique_runs = fraction_metrics['run_no'].unique()
n_runs = len(unique_runs)

# Force 11 subplots as per feedback
n_subplots = 11
fig, axes = plt.subplots(n_subplots, 1, figsize=(10, 2 * n_subplots))
if n_subplots == 1:
    axes = [axes]

# Ensure axes is a list, handling numpy array return from subplots
if isinstance(axes, np.ndarray):
    axes = axes.flatten().tolist()
elif not isinstance(axes, list):
    axes = [axes]

# Plot data for each run, up to 11
for idx, run in enumerate(unique_runs):
    if idx >= n_subplots:
        break
    ax = axes[idx]

    # Get signal data for this run
    run_signal = df_work[df_work['run_no'] == run]

    # Plot 280nm signal
    ax.plot(run_signal['volume_ml'], run_signal['UV_1_280_mAU'], color='blue',
            linewidth=0.5, label='280nmUSignal')

    # Get fraction boundaries for this run
    run_fractions = fraction_metrics[fraction_metrics['run_no'] == run]

    # Plot fraction boundaries: only min_vol for each fraction, and max_vol for
    # the last fraction
    # To avoid duplicate lines, we plot min_vol for all, and max_vol only for the
    # last fraction in the run
    min_vols = run_fractions['min_vol'].values
    max_vols = run_fractions['max_vol'].values

    # Plot min boundaries
    for vol in min_vols:
        ax.axvline(x=vol, color='gray', linestyle='--', alpha=0.5)

    # Plot max boundary only for the last fraction
    if len(max_vols) > 0:
        ax.axvline(x=max_vols[-1], color='gray', linestyle='--', alpha=0.5)

    # Highlight consecutive outlier peaks
    outlier_regions = run_fractions[run_fractions['is_consecutive_outlier']]

    # Vectorized plotting for spans
    if not outlier_regions.empty:
        min_outlier = outlier_regions['min_vol'].values
        max_outlier = outlier_regions['max_vol'].values
        for m, M in zip(min_outlier, max_outlier):
            ax.axvspan(m, M, color='red', alpha=0.3)

```

```

    ax.set_title(f'Run_{run}: Purity and Consecutive Outlier Analysis')
    ax.set_xlabel('Volume (mL)')
    ax.set_ylabel('UV280 mAU')
    ax.legend(loc='upper right', fontsize='small')

# Handle remaining subplots if n_runs < 11
for idx in range(n_runs, n_subplots):
    ax = axes[idx]
    ax.text(0.5, 0.5, 'No Data', transform=ax.transAxes, ha='center', va='center')
    ax.set_title(f'Run_{idx+1}: No Data')
    ax.set_xlabel('Volume (mL)')
    ax.set_ylabel('UV280 mAU')
    ax.axis('off')

plt.tight_layout()
plt.savefig('/workspace/all_runs_purity_outliers.png', dpi=150)
plt.close()

print("Analysis complete. Figure saved to /workspace/all_runs_purity_outliers.png")

```

Listing 3: Code_3